

BESANT TECHNOLOGIES

Department Applied for: Data Analytics | Course: Python

FILE HANDLING APPLICATION

A Mini Project Report

Submitted to: Gowthami Shyam

Submitted by: Yemuna V

Date of Submission: 07/09/2026

GitHub: <https://github.com/yemunav>

Acknowledgement

I want to thank my trainer, Gowthami Shyam, for teaching file handling and the os module clearly in class. Her step-by-step way of explaining things in Jupyter Notebook made it easier for me to follow along and then build this project on my own.

I am also thankful to Besant Technologies for giving me a structured place to learn again. Returning to studies after a long gap from work was not easy at first, and having a proper course to follow has helped me regain my confidence.

I would also like to thank my family for their patience while I practiced the same small programs again and again until they worked correctly. This project is a small step, but it means a lot to me as I work towards moving into a Data Analytics role.

Index

1. Acknowledgement.....	2
2. Introduction to Python.....	4
3. Introduction about the Project.....	6
4. Concepts Used in the Project.....	8
5. Source Code (Screenshots).....	10
6. Explanation of Source Code.....	12
7. Output (Screenshots).....	14
8. Conclusion.....	21
9. Bibliography.....	22

Introduction to Python

Python is a programming language created by Guido van Rossum and first released in 1991. It is known for being easy to read compared to many other languages, since the code closely resembles plain English. This is one reason it is often the first language taught to beginners, although it is also widely used for large, serious applications in the industry.

Python is an interpreted language, which means the code runs line by line rather than being compiled first, as with languages like C or Java. This makes it convenient for testing small pieces of code quickly, which is the approach we have followed in class — writing a small piece of code in one Jupyter Notebook cell, running it, and building on it in the next cell.

Python also does not require a variable's data type to be declared; it works this out automatically based on the value given to it. It supports more than one style of programming, including procedural and object-oriented, which makes it a flexible language for different kinds of projects.

A major advantage of Python is the number of things already built into it. Modules such as `os`, `sys`, `math` and `datetime` come with Python and do not need to be installed separately. In addition, there are a large number of external packages available (through PyPI, or already included with Anaconda) for tasks such as data analysis (`pandas`, `NumPy`), plotting (`Matplotlib`, `Seaborn`), web development (`Django`, `Flask`), and machine learning (`scikit-learn`, `TensorFlow`).

This is also why Python is so relevant to data analytics. `pandas` and `NumPy` are commonly used to clean and work with data, and `Matplotlib` or `Seaborn` are used to visualise it. This is largely why Python and SQL are considered the two main skills for a data analyst — SQL to retrieve data from a database, and Python to process it further.

Reasons Python is widely used

- Simple syntax that is easy for beginners to learn
- Runs line by line, making it convenient to test code in Jupyter Notebook
- No need to declare variable types
- Comes with many built-in modules for files, math, dates, etc.
- Works the same way on Windows, Mac and Linux
- A large number of additional libraries for data science and automation
- A large amount of documentation and community support available online

This project focuses on one of these basics: file handling, which is how a Python program creates, reads, changes or deletes files. Alongside this, it also uses the `os` module, which allows Python to interact with the operating system directly — checking the current folder, listing files in it, or moving between folders.

Introduction about the Project

This project, File Handling Application, is a menu-based Python program built and run in Jupyter Notebook through Anaconda Navigator, the same setup used in class. The aim was to bring together everything taught in the File Handling and os Module sessions into a single working program, rather than leaving each idea as a separate, one-off notebook cell.

Instead of writing each operation as its own isolated cell, every operation — create, write, read, replace, append, rename, delete — along with a few os module operations — show current directory, list files, and change directory — was written as its own function. A single menu, running inside a loop, then allows any of these operations to be carried out, as many times as needed, without restarting the program.

What the program does

When run, the program displays a numbered menu with 11 options, covering every file handling and os operation taught in class. The number entered by the user is checked using an if / elif chain, which then calls the matching function. The menu itself sits inside a while loop, so once one operation finishes, the menu is displayed again, until option 11 (Exit) is chosen. If a number outside the range 1–11 is entered, the program prints an “Invalid” message and displays the menu again rather than stopping with an error.

Why the project was built this way

A menu-driven program was chosen instead of separate scripts because it is closer to how a small real-world tool would be structured — one program offering several related actions, rather than many disconnected files. It also gave an opportunity to practice functions, conditional statements, loops and the os module together, rather than seeing each one in isolation.

Goals for this project

- Practice all three file access modes covered in class — "w" (create/replace), "a" (append), "r" (read)
- Use the os functions covered in class — getcwd(), listdir(), chdir(), rename(), unlink() — inside one working program
- Keep the code organised, with each function responsible for one task
- Keep the program limited to what has been covered so far, without additional libraries or a graphical interface

The code screenshots and output screenshots in this report are taken directly from my own notebook, `File_Handling_app.ipynb`, run on my own laptop. The file name "demo" is used for testing throughout, so the same file can be followed through create, write, read, replace, append, rename and finally delete.

Concepts Used in the Project

The concepts below are the ones from class that this project uses, along with where each one appears in the code.

1. File Handling Basics

The standard pattern for file handling in Python is to open the file using `open("filename", "mode")`, carry out an operation such as `write()` or `read()`, and then `close()` it once done. This three-step pattern is followed inside every function in this project.

2. File Access Modes

- "w" (write mode) — creates the file if it does not exist. If the file already exists, this mode replaces its old content with the new content. The same "w" mode is used for both `create()` and the separate `replace()` option, to show how it behaves differently depending on whether the file already exists.
- "a" (append mode) — creates the file if it does not exist; if it does exist, new data is added after the existing content, without deleting anything.
- "r" (read mode) — used only to read a file that already exists.

3. The os Module

The `os` module comes built into Python and allows a program to interact with the operating system. Five functions from this module are used in the project:

- `os.getcwd()` — returns the current working directory (Option 8, Show Current Directory)
- `os.listdir()` — returns a list of items in the current folder (Option 9, List Files)
- `os.chdir(path)` — changes the working directory to the given path
- `os.rename(old, new)` — renames a file
- `os.unlink(filename)` — deletes a file

4. Functions (def)

Each operation is written as its own function using `def`. Writing `def create()`: only stores the instructions; the code inside runs only when the function is actually called, for example from inside the menu. Writing the program this way made it possible to test each function individually before combining everything into the final menu.

5. input()

`input()` is used to take the file name, the data to write, or the menu choice from the user while the program is running, which is what makes the notebook behave like an interactive application rather than a set of fixed cells.

6. if / elif / else as a Menu Dispatch

Once the menu choice is read, an `if / elif / else` chain checks the number and calls the matching function — `choice==1` calls `create()`, `choice==2` calls `write()`, and so on up to `choice==11` for `Exit`. Any number outside this range falls to the final `else`, which prints the “Invalid” message. This is referred to in class as a “jump statement”, since it works in a similar way to a switch-case structure in other languages.

7. while Loop

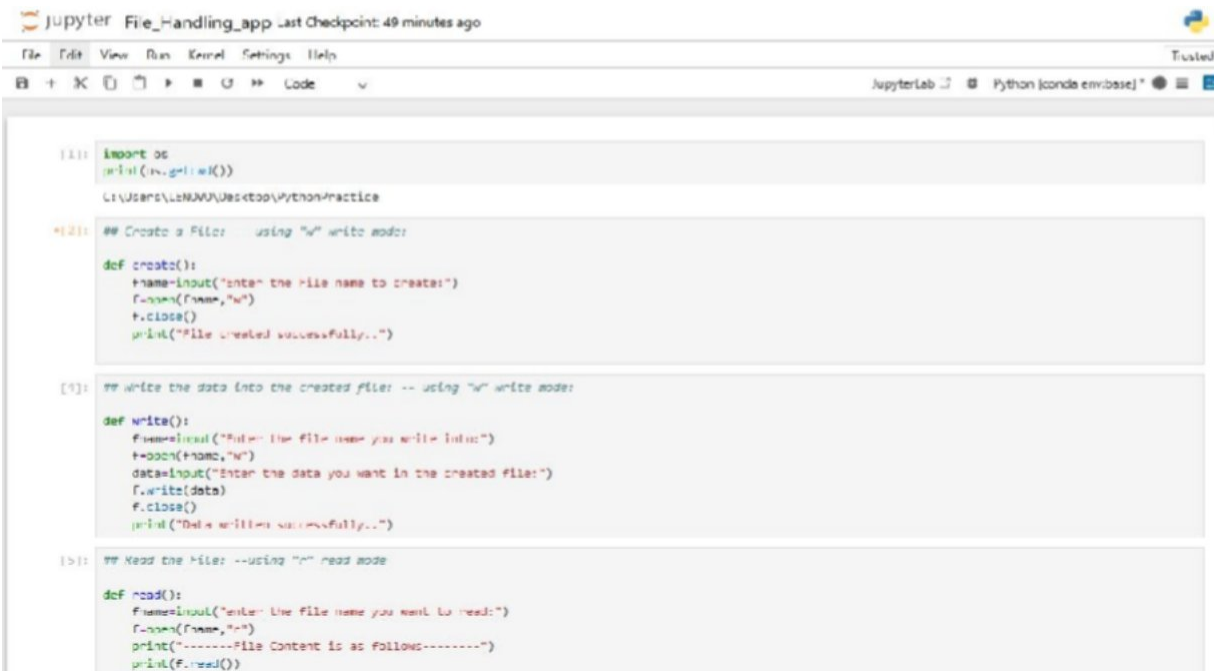
The menu is placed inside a `while True:` loop so that it continues to display until the user selects option 11, at which point a `break` statement ends the loop.

8. int() for Type Conversion

`input()` always returns text, even when a number is typed. The menu choice is therefore converted using `int(input(...))` before being compared in the `if/elif` chain, since a text value such as `"1"` is never equal to the number `1` in Python.

Source Code (Screenshots)

These are screenshots of the actual Jupyter Notebook, File_Handling_app.ipynb, taken from Anaconda Navigator, the same tool used in class. Each function was written and run in its own cell, and the final cell contains the complete menu loop.



```
jupyter File_Handling_app Last Checkpoint: 49 minutes ago
File Edit View Run Kernel Settings Help
+ K Code
JupyterLab Python [conda env:base]

[1]: import os
print(os.getcwd())
C:\Users\LENNAN\Desktop\Python\practice

[2]: ## Create a File: --using "w" write mode:

def create():
    fname=input("Enter the file name to create:")
    f=open(fname,"w")
    f.close()
    print("File created successfully..")

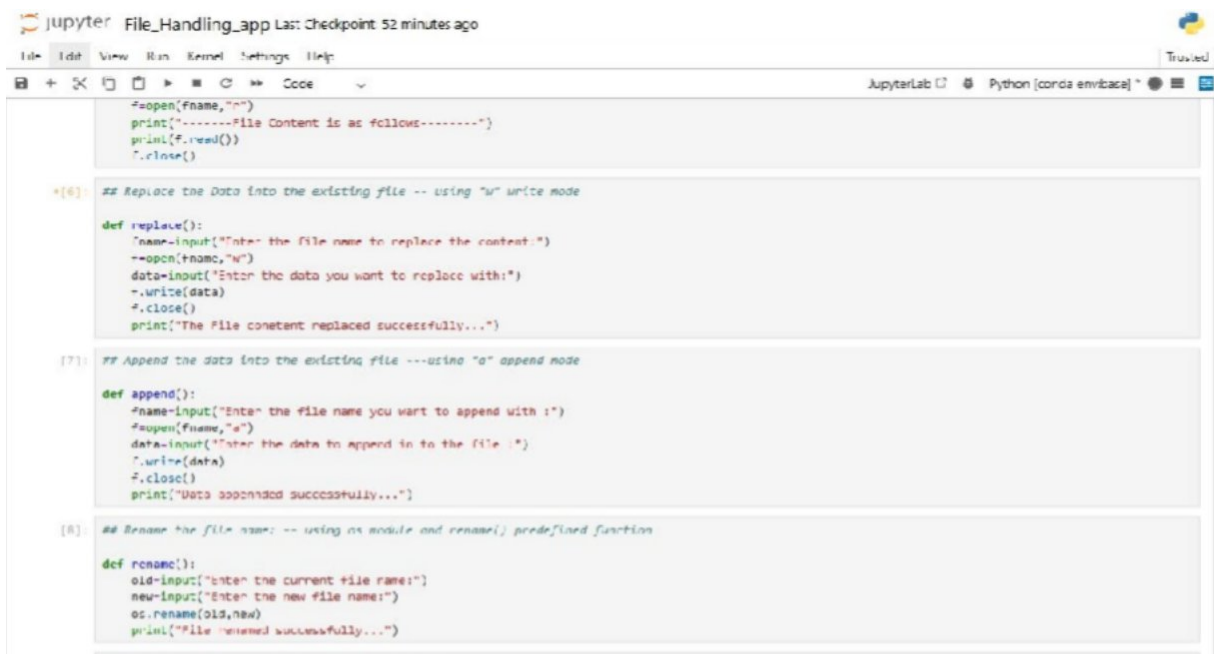
[3]: ## Write the data into the created file: -- using "w" write mode:

def write():
    fname=input("Enter the file name you will write into:")
    f=open(fname,"w")
    data=input("Enter the data you want in the created file:")
    f.write(data)
    f.close()
    print("Data written successfully..")

[4]: ## Read the file: --using "r" read mode

def read():
    fname=input("Enter the file name you want to read:")
    f=open(fname,"r")
    print("-----File Content is as follows-----")
    print(f.read())
```

Source Code — Part 1 of 5



```
jupyter File_Handling_app Last Checkpoint: 52 minutes ago
File Edit View Run Kernel Settings Help
+ K Code
JupyterLab Python [conda env:base]

[5]: f=open(fname,"r")
print("-----File Content is as follows-----")
print(f.read())
f.close()

[6]: ## Replace the Data into the existing file -- using "w" write mode

def replace():
    fname=input("Enter the file name to replace the content:")
    f=open(fname,"w")
    data=input("Enter the data you want to replace with:")
    f.write(data)
    f.close()
    print("The File content replaced successfully...")

[7]: ## Append the data into the existing file ---using "a" append mode

def append():
    fname=input("Enter the file name you want to append with:")
    f=open(fname,"a")
    data=input("Enter the data to append in to the file:")
    f.write(data)
    f.close()
    print("Data appended successfully...")

[8]: ## Rename the file name: -- using os module and rename() predefined function

def rename():
    old=input("Enter the current file name:")
    new=input("Enter the new file name:")
    os.rename(old,new)
    print("File renamed successfully...")
```

Source Code — Part 2 of 5

```

jupyter File_Handling_app Last Checkpoint: 52 minutes ago
File Edit View Run Kernel Settings Help
Python [conda env:base]

[83]: ## Rename the file name: using os module and rename() predefined function

def rename():
    old_input("Enter the current file name:")
    new_input("Enter the new file name:")
    os.rename(old,new)
    print("File renamed successfully...")

[94]: ## Delete the file -- using os.remove() predefined in module

def delete():
    name_input("Enter the file to delete:")
    os.remove(name)
    print("File deleted successfully...")

[101]: ## Show the current working directory -- using os.getcwd() predefined in module

def address():
    print("The current working directory is:")
    print(os.getcwd())

[111]: ## List all files in current directory --using listdir() predefined as module

def list():
    print("List of all files:")
    print(os.listdir())
    print("The number of files available:")
    print(len(os.listdir()))

[121]: ## Change the directory --using chdir() predefined module

def change_dir():

```

Source Code — Part 3 of 5

```

jupyter File_Handling_app Last Checkpoint: 1 hour ago
File Edit View Run Kernel Settings Help
Python [conda env:base]

[131]: ## Change the directory -- using chdir() predefined module

def change_dir():
    path_input("Enter the directory path to change to:")
    os.chdir(path)
    print("The directory changed successfully...")

[14]: ## File handling menu -- using if-else statements

while True:
    print("-----FILE HANDLING MENU-----")
    print("1. Create a File")
    print("2. Write Data")
    print("3. Read File")
    print("4. Replace File Content")
    print("5. Append to File")
    print("6. Rename File")
    print("7. Delete File")
    print("8. Show Current Directory")
    print("9. List Files in the Directory")
    print("10. Change the Directory")
    print("11. Exit")

    choice=int(input("Enter your choice:"))

    if choice==1:
        create()
    elif choice==2:
        write()
    elif choice==3:
        read()
    elif choice==4:
        replace()

```

Source Code — Part 4 of 5

```

jupyter File_Handling_app Last Checkpoint: 54 minutes ago
File Edit View Run Kernel Settings Help
Python [conda env:base]

        replace()
    elif choice==5:
        append()
    elif choice==6:
        rename()
    elif choice==7:
        delete()
    elif choice==8:
        address()
    elif choice==9:
        list()
    elif choice==10:
        change_dir()
    elif choice==11:
        print("Thank you .. Exiting the File handling app..")
        break
    else:
        print("Invalid.. Please enter the number between 1 - 11")

-----FILE HANDLING MENU-----
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Rename File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit
Enter your choice:

```

Source Code — Part 5 of 5

Explanation of Source Code

The notebook is explained below function by function.

import os

Brings in the os module so that getcwd, listdir, chdir, rename and unlink can be used later in the program.

create()

Takes a file name from the user, opens it in "w" mode and closes it immediately. Since "w" mode creates the file if it does not exist, this results in an empty file being created.

write()

Takes a file name, opens it in "w" mode, asks the user for the data to write, writes it using `f.write(data)`, and closes the file. If the file already had content, this is replaced, since "w" mode always starts from an empty file.

read()

Takes a file name, opens it in "r" mode, prints the full content using `f.read()`, and then closes the file.

replace()

Works in the same way as `write()` — it opens the file in "w" mode and writes new data. It is kept as a separate menu option since replacement was taught as a distinct concept in class, even though the underlying code is the same as `write()`.

append()

Similar to `write()`, except the file is opened in "a" mode, so the new data is added after the existing content rather than replacing it.

rename()

Takes the current file name and the new file name from the user, then calls `os.rename(old, new)`.

delete()

Takes a file name and calls `os.unlink(fname)` to delete it.

address()

Calls `os.getcwd()` and prints it, showing which folder the program is currently working in.

list()

Calls `os.listdir()` to print all files in the current folder, and `len(os.listdir())` to print how many items were found.

change_dir()

Takes a folder path from the user and calls `os.chdir(path)` to move into that folder.

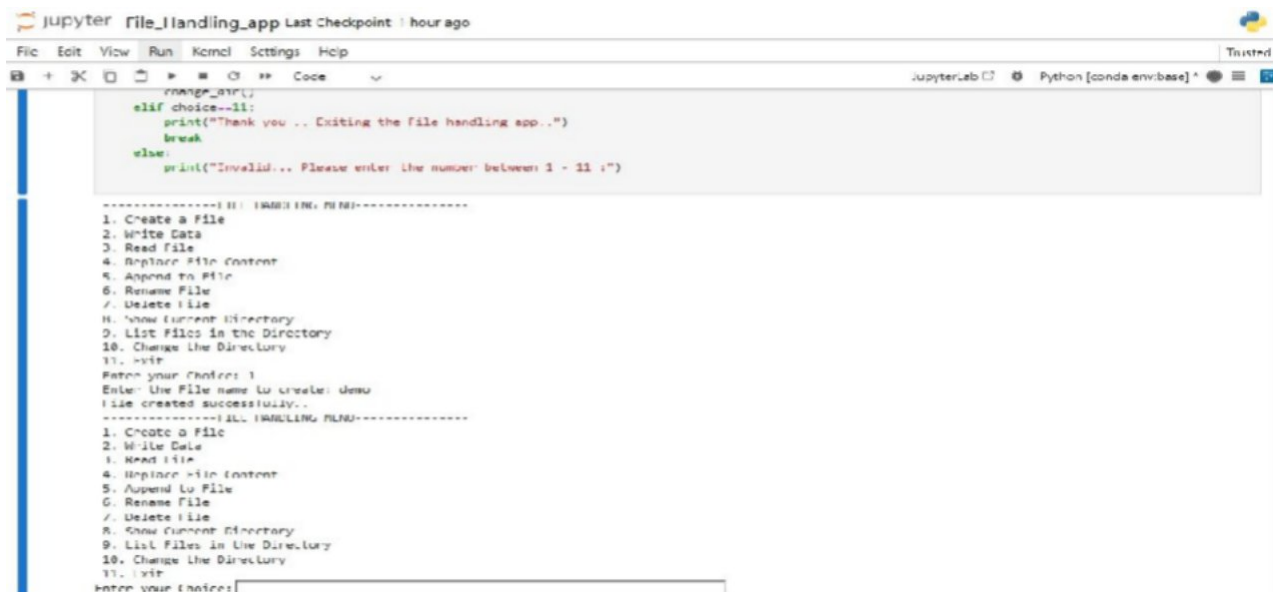
The Menu Loop

This is the `while True:` section at the end of the notebook. It displays the menu, reads the choice using `choice = int(input("Enter your Choice:"))`, and then dispatches to the matching function using `if choice==1: create()`, `elif choice==2: write()`, and so on, up to `choice==11`, which prints a closing message and uses `break` to end the loop. Any other value falls to the `else` branch, which prints an "Invalid" message and displays the menu again.

Output (Screenshots)

The screenshots below are from one continuous run of the program, working through each option in order: create a file (1), write data (2), read it back (3), replace its content (4), append to it (5), rename it (6), delete it (7), show the current directory (8), list the files (9), change the directory (10), and finally exit (11) — along with File Explorer / Notepad screenshots confirming each result on disk, and one additional run showing how an invalid menu choice is handled.

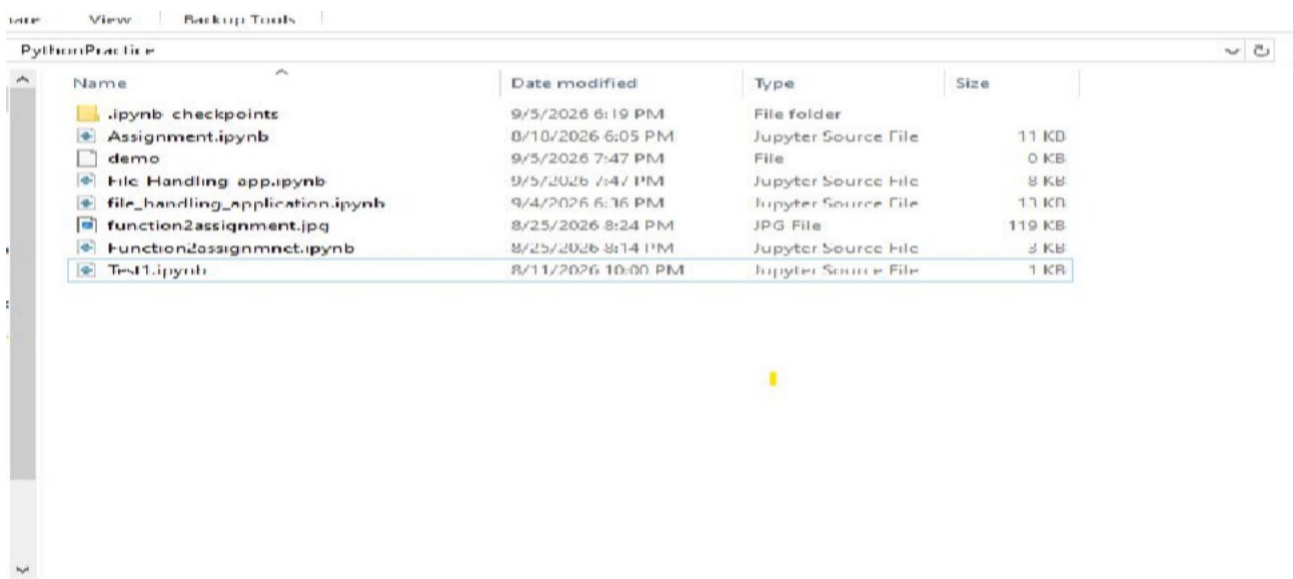
Option 1 — Create a File



```
change_dir()
elif choice==11:
    print("Thank you .. Exiting the file handling app..")
    break
else:
    print("Invalid... Please enter the number between 1 - 11 :")

-----| 11: THANK YOU, PLEASE|-----
1. Create a File
2. Write Data
3. Read file
4. Replace file content
5. Append to File
6. Rename File
7. Delete file
8. Show current directory
9. List files in the Directory
10. Change the Directory
11. Exit
Enter your choice: 1
Enter the File name to create: demo
file created successfully..
-----| 11: THANK YOU, PLEASE|-----
1. Create a File
2. Write Data
3. Read file
4. Replace file content
5. Append to File
6. Rename File
7. Delete file
8. Show current directory
9. List files in the Directory
10. Change the Directory
11. Exit
Enter your choice: 
```

Running Option 1: the file "demo" is created.



Name	Date modified	Type	Size
.ipynb checkpoints	9/5/2026 6:19 PM	File folder	
Assignment.ipynb	8/10/2026 6:05 PM	Jupyter Source File	11 KB
demo	9/5/2026 7:47 PM	File	0 KB
File Handling app.ipynb	9/5/2026 7:47 PM	Jupyter Source File	8 KB
file_handling_application.ipynb	9/4/2026 6:36 PM	Jupyter Source File	11 KB
function2assignment.jpg	8/25/2026 8:24 PM	JPG File	119 KB
function2assignment.ipynb	8/25/2026 8:14 PM	Jupyter Source File	8 KB
Test1.ipynb	8/11/2026 10:00 PM	Jupyter Source File	1 KB

File Explorer confirms "demo" now exists (0 KB, empty file).

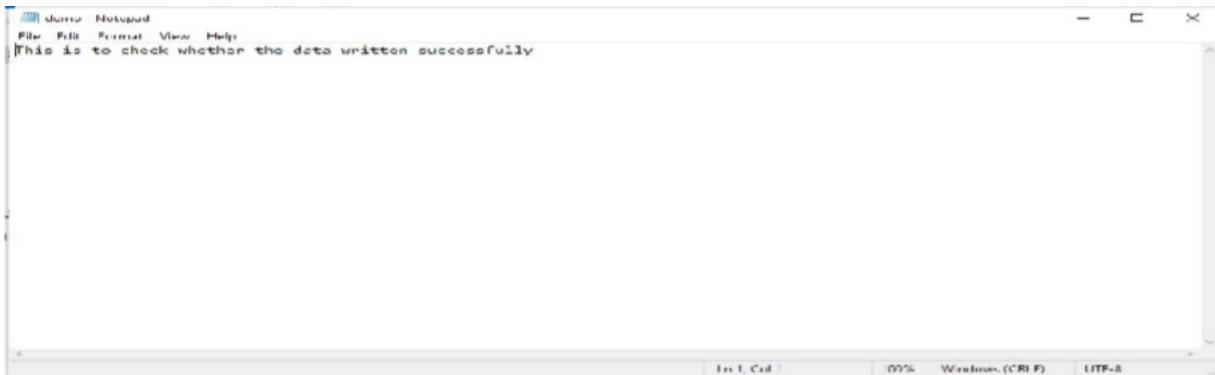
Option 2 — Write Data



```
jupyter File_Handling_app Last Checkpoint: 1 hour ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base]

Enter your Choice: 1
Enter the file name to create: demo
File created successfully..
-----FILE HANDLING MENU-----
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Remove File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit
Enter your Choice: 2
Enter the file name you want to write into: demo
Enter the data you want in the created file: This is to check whether the data written successfully
Data written successfully..
-----FILE HANDLING MENU-----
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Remove File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit
Enter your Choice: 
```

Running Option 2: data is written into “demo”.



```
demo Notepad
File Edit Format View Help
This is to check whether the data written successfully
```

Notepad confirms the written content inside “demo”.

Option 3 — Read File



```
jupyter File_Handling_app Last Checkpoint: 1 hour ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base]

9. List Files in the Directory
10. Change the Directory
11. Exit
Enter your Choice: 2
Enter the file name you want to write into: demo
Enter the data you want in the created file: This is to check whether the data written successfully
Data written successfully..
-----FILE HANDLING MENU-----
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Remove File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit
Enter your Choice: 3
Enter the file name you want to read from: demo
-----File Content is as follows-----
This is to check whether the data written successfully
-----FILE HANDLING MENU-----
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Remove File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit
Enter your Choice: 
```

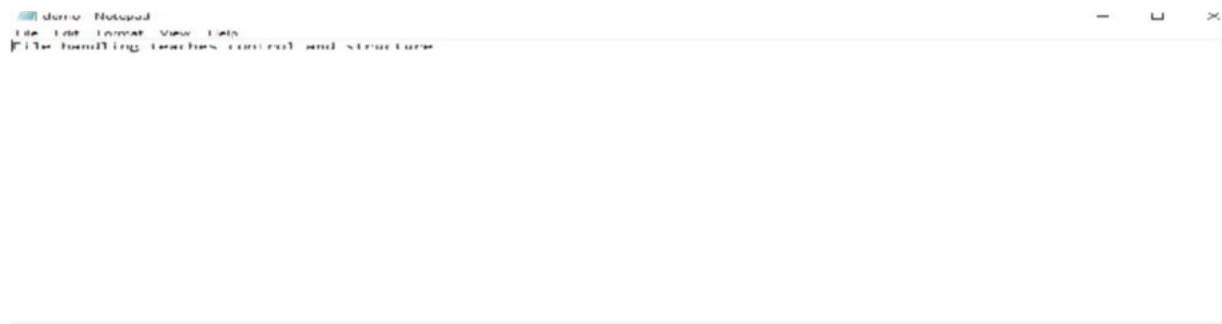
Running Option 3: the content of “demo” is printed on screen.

Option 4 — Replace File Content



The screenshot shows a JupyterLab window titled "File_Handling_app" with a last checkpoint of 1 hour ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The main area displays a menu for "File_Handling_app" with the following options: 1. Create a File, 2. Write Data, 3. Read File, 4. Replace File Content, 5. Append to File, 6. Rename File, 7. Delete File, 8. Show Current Directory, 9. List Files in the Directory, 10. Change the Directory, 11. Exit. The user has selected Option 4, "Replace File Content". The application prompts the user to enter the file name to replace the content, with "demo" entered. It then prompts for the data to replace with, with "file handling teaches control and structure" entered. A confirmation message states: "The File content replaced successfully...". The application then displays a separator line "*****FILE HANDLING MENU*****" and the main menu again. The user is prompted to enter their choice, with "4" entered.

Running Option 4: the content of "demo" is replaced.



The screenshot shows a Notepad window titled "demo - Notepad" with a menu bar (File, Edit, Format, View, Help). The text content of the file is "File handling teaches control and structure".

Notepad confirms the old content is gone and new content is in place.

Option 5 — Append to File



The screenshot shows a JupyterLab window titled "File_Handling_app" with a last checkpoint of 1 hour ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The main area displays a menu for "File_Handling_app" with the following options: 1. Create a File, 2. Write Data, 3. Read File, 4. Replace File Content, 5. Append to File, 6. Rename File, 7. Delete File, 8. Show Current Directory, 9. List Files in the Directory, 10. Change the Directory, 11. Exit. The user has selected Option 5, "Append to File". The application prompts the user to enter the file name to append with, with "demo" entered. It then prompts for the data to append to the file, with "Every project begins with a file" entered. A confirmation message states: "Data appended successfully...". The application then displays a separator line "*****FILE HANDLING MENU*****" and the main menu again. The user is prompted to enter their choice, with "5" entered.

Running Option 5: new data is appended to "demo".

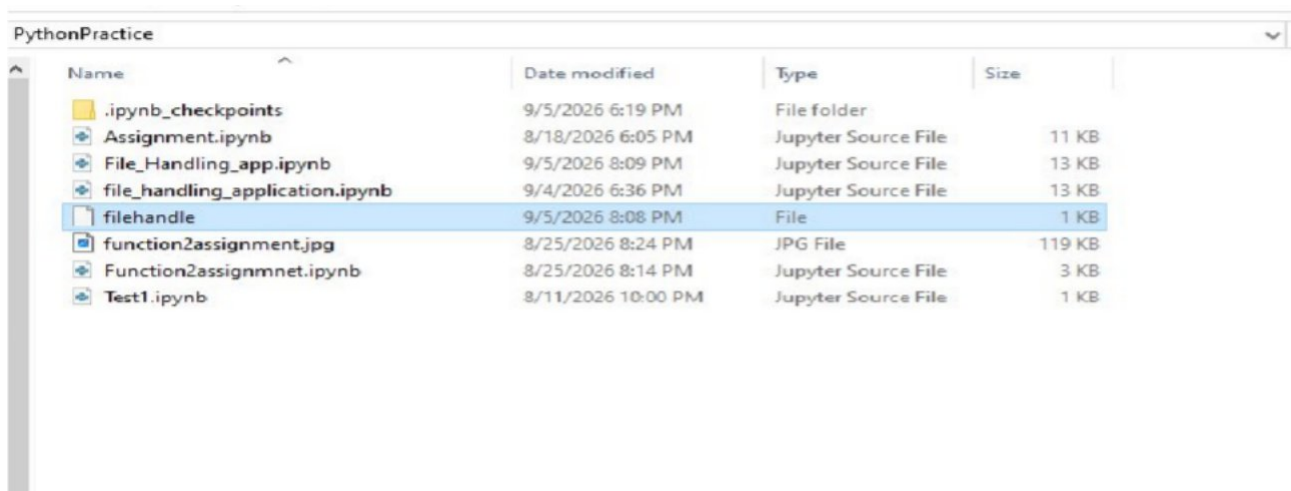


Notepad confirms the new sentence was added after the existing content.

Option 6 — Rename File

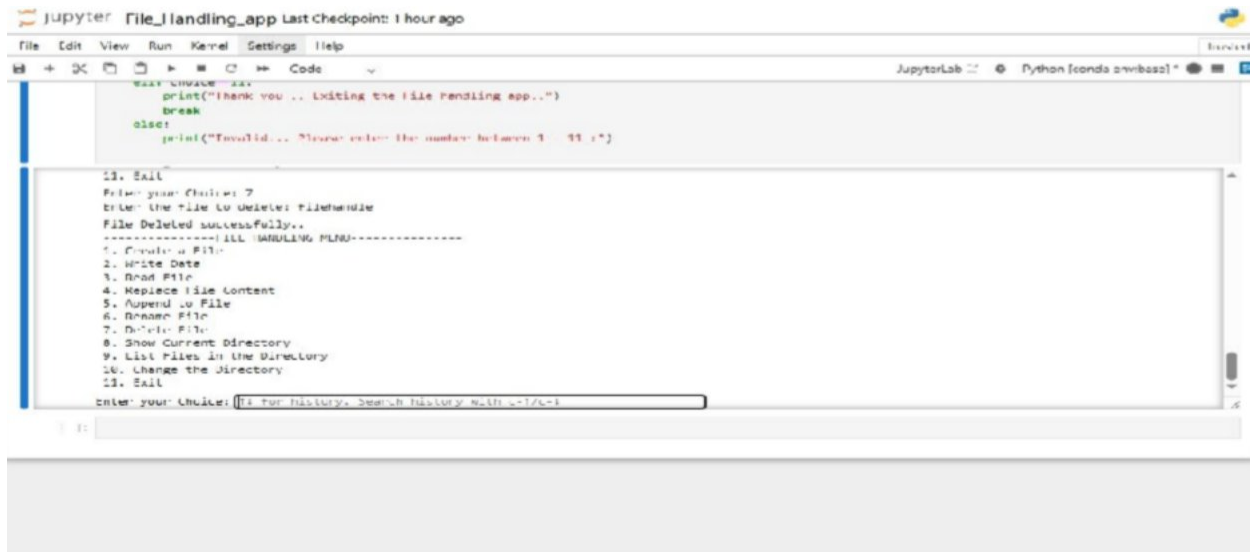


Running Option 6: “demo” is renamed to “filehandle”.



File Explorer confirms the file now appears as “filehandle”.

Option 7 — Delete File

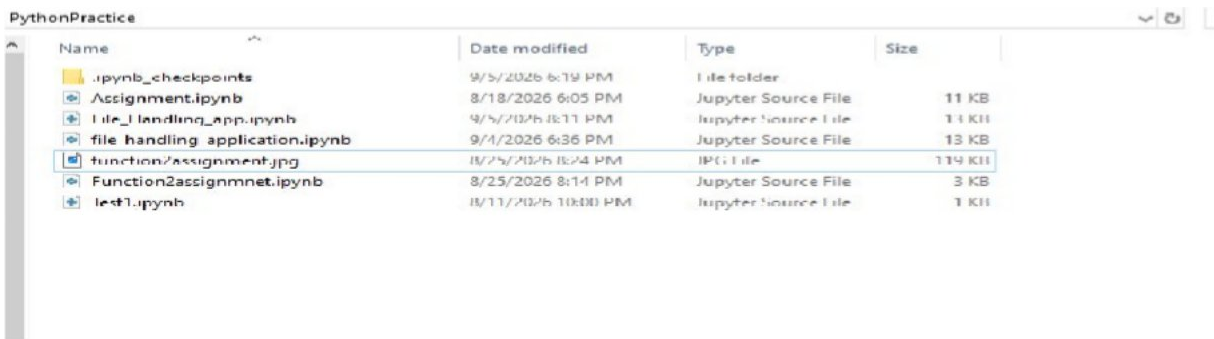


```
File Handling App
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Remove File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit

Enter your Choice: 7
Enter the file to delete: filehandle
File Deleted successfully..
-----WELCOME TO THE FILE HANDLING APP-----
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Remove File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit

Enter your Choice: 11
```

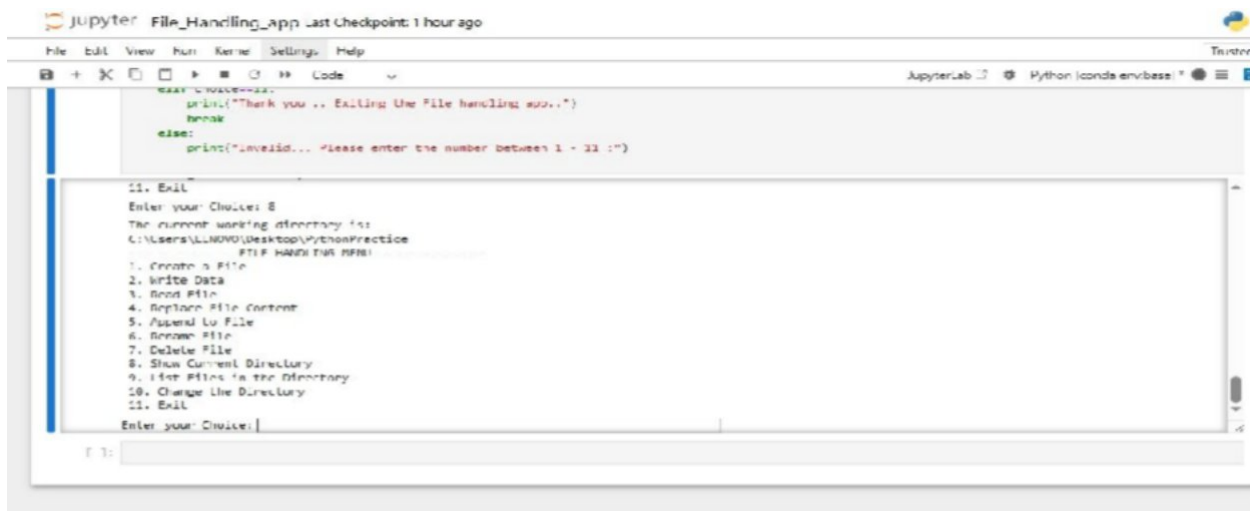
Running Option 7: "filehandle" is deleted.



Name	Date modified	Type	Size
.ipynb_checkpoints	9/5/2026 6:19 PM	File folder	
Assignment.ipynb	8/18/2026 6:05 PM	Jupyter Source File	11 KB
File_Handling_App.ipynb	9/5/2026 6:17 PM	Jupyter Source File	11 KB
file_handling_application.ipynb	9/1/2026 6:36 PM	Jupyter Source File	13 KB
function/assignment.jpg	10/25/2026 10:24 PM	Image File	119 KB
function2assignment.ipynb	8/25/2026 8:14 PM	Jupyter Source File	3 KB
test1.ipynb	10/11/2026 10:00 PM	Jupyter Source File	1 KB

File Explorer confirms "filehandle" no longer exists in the folder.

Option 8 — Show Current Directory



```
File Handling App
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Remove File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit

Enter your Choice: 8
The current working directory is:
C:\Users\LLH\OneDrive\PythonPractice
(Python 3.11.0)

Enter your Choice: 11
```

Running Option 8: the current working directory is printed.

Option 9 — List Files in the Directory



```
File Handling app Last Checkpoint: 1 hour ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base]

def main():
    print("Thank you .. Exiting the File Handling app..")
    break
else:
    print("Invalid... Please enter the number between 1 - 11 :")

# List of all files:
files = ['Assignment.ipynb', 'File_Handling_app.ipynb', 'File_Handling_Application.ipynb', 'FunctionAssignment.jpg', 'FunctionAssignment.ipynb', 'Test1.ipynb']

# The number of files available:
count = len(files)

#-----FILE HANDLING MENU-----
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Rename File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit

Enter your choice: 9
List of all files:
['Assignment.ipynb', 'File_Handling_app.ipynb', 'File_Handling_Application.ipynb', 'FunctionAssignment.jpg', 'FunctionAssignment.ipynb', 'Test1.ipynb']
The number of files available:
6
-----FILE HANDLING MENU-----
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Rename File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit
```

Running Option 9: all files in the current directory are listed, along with the count.

Option 10 — Change the Directory



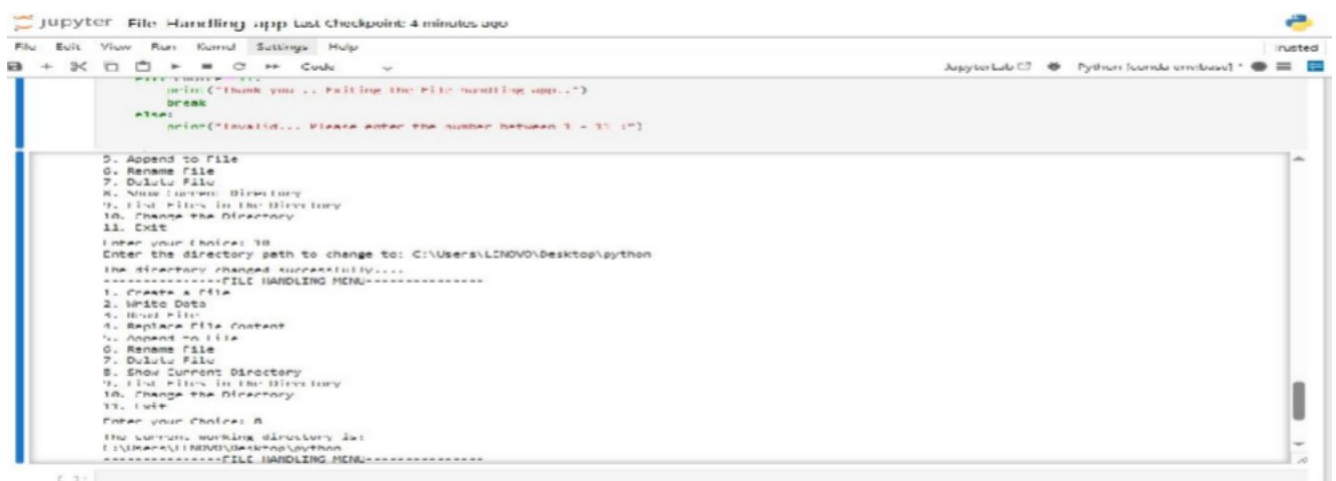
```
File Handling app Last Checkpoint: 2 minutes ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base]

def main():
    print("Thank you .. Exiting the File Handling app..")
    break
else:
    print("Invalid... Please enter the number between 1 - 11 :")

#-----FILE HANDLING MENU-----
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Rename File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit

Enter your choice: 10
Enter the directory path to change to: C:\Users\LENOVO\Desktop\python
The directory changed successfully....
-----FILE HANDLING MENU-----
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Rename File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit
```

Running Option 10: the working directory is changed to a new path.



```
File Handling app Last Checkpoint: 4 minutes ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base]

def main():
    print("Thank you .. Exiting the File Handling app..")
    break
else:
    print("Invalid... Please enter the number between 1 - 11 :")

#-----FILE HANDLING MENU-----
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Rename File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit

Enter your choice: 8
The current working directory is:
C:\Users\LENOVO\Desktop\python
-----FILE HANDLING MENU-----
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Rename File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit
```

Choosing Option 8 again afterwards confirms the directory actually changed.

The screenshot shows a Jupyter Notebook window titled "File_Handling_app" with a last checkpoint 6 minutes ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar. The code cell contains a Python script for a file handling application. The output cell shows the application's execution, displaying a menu and processing user input.

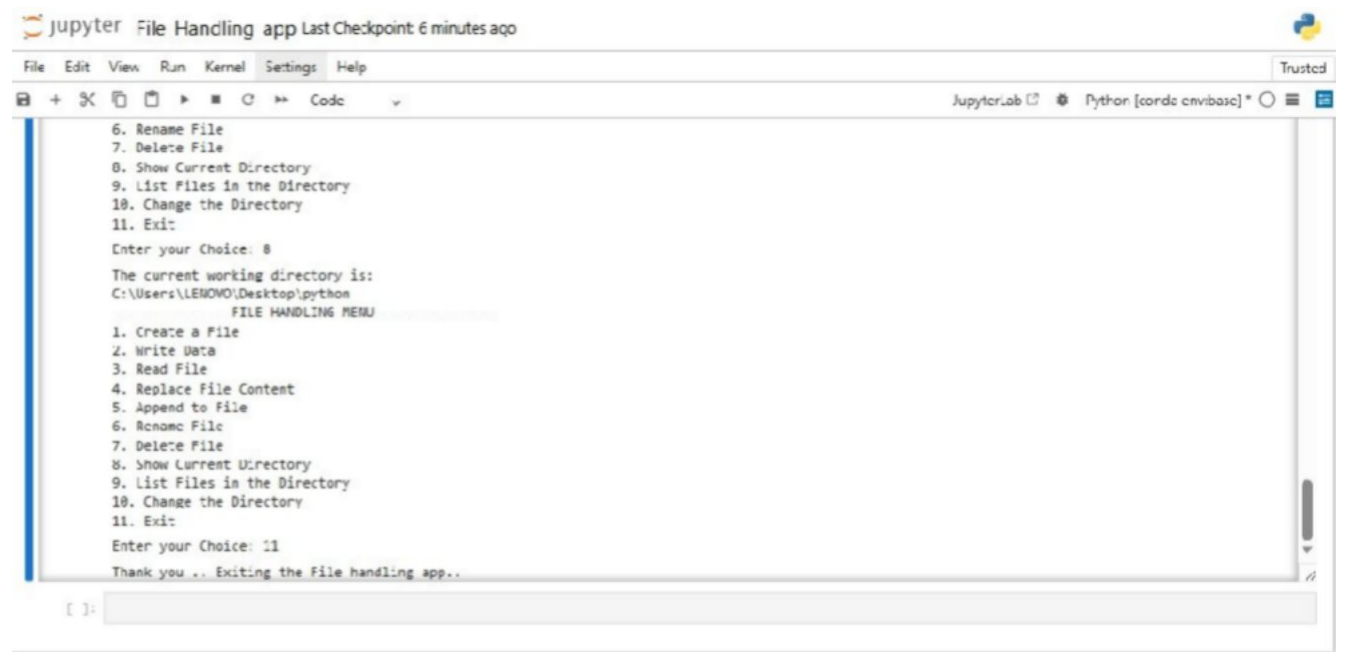
```

elif choice==10:
    change_dir()
elif choice==11:
    print("Thank you .. Exiting the File handling app..")
    break
else:
    print("Invalid... Please enter the number between 1 - 11 :")

FILE HANDLING MENU
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Rename File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit
Enter your Choice: 14
Invalid... Please enter the number between 1 - 11 :
FILE HANDLING MENU
1. Create a File
2. Write Data
3. Read File
4. Replace File Content
5. Append to File
6. Rename File
7. Delete File
8. Show Current Directory
9. List Files in the Directory
10. Change the Directory
11. Exit
Enter your Choice:

```

Option 11 — Exit



20

Conclusion

This project brought together everything covered in the File Handling and os Module classes into one working menu-based program in Jupyter Notebook. Create, write, read, replace, append, rename and delete were all implemented, along with `getcwd()`, `listdir()` and `chdir()`, tied together behind a single menu that continues running until the user chooses to exit.

Beyond file handling itself, the project was also useful for learning how to structure a program — breaking it into small functions, testing each one individually before adding it to the menu, and working through a genuine bug along the way: the menu choice initially was not converted with `int()`, which meant valid choices were incorrectly treated as invalid until this was corrected.

Overall, this project has built confidence in writing Python independently in Jupyter Notebook, and provides a reasonable base to build on. The next steps planned are working with CSV files, and later pandas, alongside the SQL already covered.

Bibliography

- Python Official Documentation — <https://docs.python.org/3/>
- Python File Handling Tutorial — <https://docs.python.org/3/tutorial/inputoutput.html>
- Python os module documentation — <https://docs.python.org/3/library/os.html>
- Jupyter Notebook Documentation — <https://docs.jupyter.org/>
- Anaconda Navigator Documentation — <https://docs.anaconda.com/navigator/>
- GeeksforGeeks — Python File Handling — <https://www.geeksforgeeks.org/file-handling-python/>
- W3Schools — Python File Handling — https://www.w3schools.com/python/python_file_handling.asp
- Class notes and the File Handling and os Module notebook provided by Gowthami Shyam, Besant Technologies.
- Project source code and notebook: <https://github.com/yemunav>